# Rapport complet de développement et d'audit
# eLLM_v1

Rapport scientifique, opérationnel et expérimental

Version 1.0 – 21 octobre 2026

**Résumé**

Ce rapport consigne l'analyse des sources originales, l'architecture implémentée, le pipeline logiciel, les essais exécutés, leurs résultats et les écarts non résolus. La source théorique principale est le document DDD_Base transmis avec les datasets. Le code v1 est structuré autour d'un modèle causal préentraîné, d'un adaptateur de sondes apprises multi-résolution qui affecte les logits, d'un pipeline d'import et de fine-tuning, d'un historique SQLite et d'une interface Web locale. Les tests de composants et du serveur ne remplacent pas l'évaluation avec les poids réels : le téléchargement du checkpoint a échoué dans l'environnement de construction, ce qui est documenté explicitement.

## Table des matières

**1 Description** **2**
- 1.1 Objectif . . . 2
- 1.2 Sources étudiées . . . 2
- 1.3 Inventaire des datasets . . . 4

**2 Architecture** **5**
- 2.1 Différence essentielle avec eLLM_v0 . . . 5
- 2.2 Choix du checkpoint . . . 5
- 2.3 Modules . . . 5

**3 Pipeline complet** **6**

**4 Commandes d'exécution** **6**

**5 Résultats des essais exécutés** **7**
- 5.1 Essais de code et théorie finie . . . 7
- 5.2 Données . . . 7
- 5.3 Essai réel de téléchargement . . . 7
- 5.4 Interface . . . 7
- 5.5 Import d'un checkpoint déjà présent sur le PC utilisateur . . . 7
- 5.6 Validation locale P07 et protocole de fine-tuning . . . 8
- 5.7 Expérience discriminante de résolution (P06) . . . 8

**6 Interprétation des résultats** **9**

**7 Conséquences théoriques, conceptuelles et opérationnelles** **9**
- 7.1 Théoriques . . . 9
- 7.2 Conceptuelles . . . 9
- 7.3 Opérationnelles . . . 9

8 Conclusions 9

9 Statut du modèle 10

# 1 Description

## 1.1 Objectif

Le projet vise à mettre en œuvre un modèle de langage local qui utilise les notions de la théorie des empreintes comme éléments actifs de ses représentations et transformations, plutôt que d'ajouter seulement un observateur suffixe à un prédicteur distinct. eLLM_v1 doit permettre le chat local, l'historique, le chargement de datasets, le mapping des champs, le fine-tuning, l'analyse d'empreintes, la recherche et l'imagination en conservant la provenance et les échecs.

## 1.2 Sources étudiées

Les sources primaires sont archivées dans le projet : `DDD_Base_theorie_des_empreintes_complete_order_5.t` une archive `resources_0.zip` et quatre jeux de données. DDD_Base contient 3 922 lignes et 180 titres de partie/chapitre/section recensés. Les concepts pertinents couvrent objets/contexte/sondes/-fonctionnel, échelles emboîtées, indiscernabilité/fibres/quotient, fermeture/factorisation, seuils/bords, géométrie, trajectoires, généralisation observationnelle, sondes apprises, méta-observation, déficit, contraintes, provenance, recherche et imagination conservatrice. Le fichier de correspondance associe ces concepts à leur emplacement source et au code v1.

### 1.3 Inventaire des datasets

| Source | Volume observé | Statut d'import dans l'environnement |
|---|---|---|
| `Screenplay_dataset.jsonl` | 4 000 lignes valides | JSONL décodé ; champs de conversation, métadonnées et langue présents. |
| `batch_dataset.jsonl` | 783 lignes valides + 1 invalide | L'erreur à la ligne 784 est conservée avec numéro et hash ; l'original n'a pas été réécrit. |
| `smsspamcollection.zip` | 5 574 messages UCI + 5 572 lignes augmentées | CRC validé ; un membre historique sans extension est lu comme TSV sans entête, l'autre comme CSV ; total 11 146 lignes valides observées. |
| `0000.parquet` | 7 473 lignes décodées, colonnes `question`/`answer` | Le lecteur Parquet léger a décodé les 8 groupes, contrôlé le SHA-256 source et validé l'upload/aperçu HTTP. Le fichier source est conservé sans modification et copié byte-identique dans le runtime. |

Les données fournies restent intactes. La licence ou le droit d'utilisation de chacun des jeux de données fournis par l'utilisateur n'a pas été certifié par le code ; l'opérateur doit le vérifier avant une redistribution du modèle entraîné.

# 2 Architecture

## 2.1 Différence essentielle avec eLLM_v0

La v0 utilisait un prédicteur n-gramme et des signatures suffixes principalement diagnostiques. La v1 déclare un modèle causal préentraîné compatible Transformers comme backend et apprend un ensemble de sondes depuis ses états cachés contextualisés. Les sondes sont emboîtées et une projection de ces signatures ajoute une correction aux logits. Les poids du modèle ne sont pas remplacés automatiquement par un modèle de démonstration ou aléatoire.

## 2.2 Choix du checkpoint

Le checkpoint par défaut est `HuggingFaceTB/SmolLM2-135M-Instruct`, épinglé à la révision immuable `ee72e5415c002e8fc0566a191a07c0b2b708875a`. Sa fiche officielle indique 135 millions de paramètres et une licence Apache-2.0 ; la page officielle du fichier annonce le SHA-256 `5af571cbf074e6d21a03528d233` `https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct/blob/ee72e5415c002e8fc0566a191a07c0` `model.safetensors`. La sélection est configurable. Le chargeur local vérifie configuration et poids, produit un manifeste SHA-256 et refuse les fichiers incomplets. Les poids eux-mêmes ne sont pas incorporés au package, car le téléchargement a échoué dans l'environnement courant lors d'une tentative vers le dépôt officiel. Aucune inference ni entraînement avec ces poids ne sont donc revendiqués comme accomplis.

## 2.3 Modules

| Module | Fonction opérationnelle |
|---|---|
| `pretrained.py`, `manager.py` | Téléchargement/reprise et inventaire du checkpoint, chargement explicite, refus de fallback, gestion des adaptateurs. |
| `fingerprint_adapter.py`, `model.py` | Sondes apprises sur états cachés, résolutions préfixes, signatures quantifiées, correction de logits, traces. |
| `theory.py` | Contextes, signatures, partition, raffinement, fermeture, factorisation, capacités et dimension effectives, seuils et bord, pseudométriques, transport et contraintes. |
| `dataio.py` | Import JSONL/JSON/CSV/TSV/TXT/Markdown/ZIP/Parquet, mapping, hashes, erreurs de lignes, création d'exemples et split regroupé par doublons exacts. |
| `trainer.py` | Fine-tuning LoRA ou complet, loss causale et multi-résolution, sélection par validation, checkpoints de récupération, test après sélection et ablation empreinte active/inactive. |
| `history.py` | Historique conversationnel SQLite, liste/recherche/renommage/suppression/export. |
| `search.py`, `engine.py`, `diagnostics.py` | Recherche par faisceau, imagination échantillonnée et reproductible, révision de candidat sous contraintes publiques, déficits opérationnels. |
| `server.py`, `web/` | API HTTP locale et interface responsive avec chat, historique, upload, mapping, réglages d'entraînement, état/progression des jobs, empreintes et expériences. |

# 3 Pipeline complet

1. **Inventaire et provenance** : préserver l'archive et les originaux, SHA-256, CRC, taille, format et métadonnées accessibles.
2. **Installation du moteur** : installer les bibliothèques dans l'environnement utilisateur, télécharger le checkpoint choisi, vérifier chaque poids et écrire le manifeste.
3. **Import des données** : décoder le format déclaré, afficher colonnes/exemples, conserver l'original, rapporter les lignes invalides et mapper messages/texte/prompt/cible/label/ID.
4. **Normalisation des exemples** : transformer messages role/content, prompt-cible ou texte causal en exemples entraînables. Ne jamais injecter une réponse attendue qui n'était pas dans la donnée sélectionnée.
5. **Partition** : grouper les hashes exacts identiques avant train/validation/test et enregistrer les hashes de chaque split. Cela prévient les doublons exacts trans-partitions, mais ne prouve pas l'absence de similarité sémantique.
6. **Empreinte initiale** : le backbone causal produit états cachés contextualisés. Les groupes de sondes apprises construisent des familles emboîtées ; l'égalité exacte se rapporte à la signature quantifiée déclarée.
7. **Apprentissage** : entraînement LoRA ou complet, gradient de la perte LM et des prédictions multi-résolution, régularisation de redondance de sondes, checkpoints périodiques et journaux.
8. **Sélection** : choisir le meilleur epoch selon NLL de validation. Le test réservé ne doit pas choisir l'hyperparamètre ni l'epoch.
9. **Évaluation** : mesurer le modèle adapté, l'ablation empreinte désactivée à poids identiques et une baseline préentraînée rechargée, après sélection.
10. **Chat et outils** : formater l'historique, générer localement, conserver la provenance, proposer exploration/imagination, puis exécuter des contrôles explicites sur les contraintes publiques.
11. **Audit** : relier claims à sources, symboles, lignes, tests, hashes et rapports ; garder les résultats négatifs et incidents dans les checklists.

# 4 Commandes d'exécution

Les commandes suivantes sont des commandes de référence pour une machine avec Python 3.10+ et un accès réseau lors de l'installation/téléchargement :

```
python setup_eLLM_v1.py
python run_eLLM_v1.py status
python run_eLLM_v1.py download-model --model-id HuggingFaceTB/SmolLM2-135M
   -Instruct --revision ee72e5415c002e8fc0566a191a07c0b2b708875a
python run_eLLM_v1.py serve --host 127.0.0.1 --port 8765
python run_eLLM_v1.py selftest -v
```

Le site local est ouvert à `http://127.0.0.1:8765`. Dans Réglages, le contrôle `Importer le checkpoint local` permet de saisir le chemin absolu du dossier de poids situé sur la même machine que le serveur. L'import passe par un staging distinct, contrôle la structure et les empreintes SHA-256, conserve une copie précédente en cas de remplacement explicite et ne contourne jamais les contrôles de provenance du checkpoint épinglé. Dans la vue Entraînement, l'utilisateur sélectionne ou charge un dataset, vérifie l'aperçu et le mapping, configure les hyperparamètres puis lance et suit un job. Le chat utilise `sessions/history.sqlite3`. L'environnement d'audit a effectué les tests unitaires et les tests HTTP sans checkpoint ; il ne peut pas générer de réponse du vrai modèle tant que ses poids ne sont pas installés.

# 5 Résultats des essais exécutés

## 5.1 Essais de code et théorie finie

La suite Python a passé 103/103 tests lors de la régression complète P07 du 21 octobre 2026 (journal brut : `phase_logs/P07-full-unittest-regression.log`, agrégat : `phase_logs/P07-local-checkpoint-har`
Elle couvre l'import (JSONL avec ligne invalide, conversations, mapping, doublons, ZIP SMS et lecture Parquet du fichier fourni), la théorie (signature emboîtée, partition/raffinement, seuil et bord, fermeture, factorisation, capacité/dimension, transitions, commutation, contraintes et expérience sur fibres non triviales), l'adaptateur (correction de logits et gradients, quantification aux frontières positives et négatives), l'entraîneur, l'historique persistant, l'intégrité du checkpoint et les routes HTTP. Cette suite valide les contrats des composants et le protocole fini, pas l'inférence avec les poids externes.

## 5.2 Données

La ligne JSONL corrompue n'est pas supprimée des fichiers sources. Le test confirme le SHA du fichier inchangé après l'import, le numéro de ligne et le hash de l'erreur sont rapportés. Le ZIP SMS est traité membre par membre avec fallback explicite pour les schémas mixtes `label`/`labels`. Le lecteur Parquet portable intégré a décodé le fichier fourni : 7 473 lignes, colonnes `question`/`answer`, 8 groupes Snappy ; l'upload multipart HTTP, l'aperçu et le mapping sont testés. PyArrow reste optionnel pour les variantes Parquet hors du sous-ensemble pris en charge.

## 5.3 Essai réel de téléchargement

Plusieurs tentatives de téléchargement ont été réalisées, notamment sur la révision officielle immuable `ee72e5415c002e8fc0566a191a07c0b2b708875a`. La page officielle du fichier annonce le SHA-256 `5af571cbf074e6d21a03528d2330792e532ca608f24ac70a143f6b369968ab8c` ; les requêtes du conteneur ont échoué sur le réseau et aucun poids n'a été créé. Les événements et leurs sorties sont conservés dans `phase_logs/P04-pinned-revision-download-attempt-1.json` et `phase_logs/P04-checkpoint-`
Par conséquent, les tests de génération et de fine-tuning utilisant le checkpoint de production sont **non exécutés**, et aucune perplexité v1 sur les données de test n'est déclarée. La baseline n-gramme v0 n'est pas utilisée comme backend de repli.

## 5.4 Interface

La syntaxe JavaScript est contrôlée, les références littérales d'IDs sont comparées avec le HTML et les routes HTTP sont testées par une instance locale. Le parcours d'import local a été ajouté au test Chromium ; l'API est simulée dans le navigateur de test, tandis que la route réelle est testée indépendamment par les tests HTTP. Le test Chromium indépendant a passé 13/13 contrôles avec API simulée : chat persistant même quand le checkpoint est absent, invalidation de l'aperçu lorsqu'un mapping change, envoi des hyperparamètres, événements de job, historique, thème et navigation mobile ; aucun message console/erreur de page ni débordement horizontal à 390 px n'a été observé dans ce test. Un smoke HTTP sur une instance réelle a également importé un checkpoint synthétique non-default via `/api/model/import-local`, publié un manifeste SHA-256, puis confirmé `checkpoint_ready=true` et `loaded=false` lorsque Transformers est absent. Cette expérience vérifie la route, pas le chargement de vrais poids. Les routes HTTP ont été testées séparément sur le serveur réel. Ces contrôles n'équivalent pas à une génération avec le checkpoint officiel, qui n'est pas disponible dans l'environnement.

## 5.5 Import d'un checkpoint déjà présent sur le PC utilisateur

L'utilisateur a indiqué posséder déjà SmolLM2 sur son propre PC. Le chemin du système de fichiers de ce PC n'est pas monté dans l'environnement de construction : aucun fichier local de

l'utilisateur n'a été inspecté ni chargé ici. Pour éviter d'imposer un nouveau téléchargement, la vue Réglages permet maintenant de saisir le chemin absolu du dossier de checkpoint lorsque le serveur tourne sur la même machine. L'import copie les fichiers en staging, vérifie le contenu, écrit un manifeste, publie atomiquement le snapshot et exige une confirmation explicite avant remplacement ; l'ancienne copie est archivée. Pour l'identifiant et la révision officiels épinglés, le SHA-256 du poids principal doit correspondre au hash officiel. Le parcours est couvert par les tests d'intégrité, HTTP et navigateur avec fixtures. Cela valide le chemin d'import, pas le chargement des poids de l'utilisateur ni le fine-tuning réel, qui reste à exécuter sur sa machine.

### 5.6 Validation locale P07 et protocole de fine-tuning

Le checkpoint SmolLM2 est déjà présent sur le PC de l'utilisateur, mais son système de fichiers n'est pas monté dans le conteneur qui produit ce rapport. Pour rendre la vérification reproductible sur la machine concernée, le runtime contient maintenant `tools/validate_local_checkpoint_eLLM_v1.py`. Le script ne télécharge pas de poids, ne fait aucune installation implicite et écrit un rapport JSON même lorsque les dépendances ou la provenance sont incorrectes. Il vérifie la provenance locale, charge le modèle avec Transformers, exécute une génération courte et, si `-train` est fourni, entraîne sur le jeu sélectionné, vérifie l'intégrité du dataset et rapporte les mesures du test réservé après sélection par validation.

Les sources des datasets fournis sont livrées byte-identiques sous `training_data/provided_sources/` ; `SOURCE_DATASET_MANIFEST.json` contient leur taille et leur SHA-256. Des exemples de configuration et de mapping sont dans `examples/`. L'exécution sur le PC réel suit :

```
python setup_eLLM_v1.py
python tools/validate_local_checkpoint_eLLM_v1.py \\
  --checkpoint-dir "C:/Models/SmolLM2-135M-Instruct" \\
  --root "./ellm_v1_data"
python tools/validate_local_checkpoint_eLLM_v1.py \\
  --checkpoint-dir "C:/Models/SmolLM2-135M-Instruct" \\
  --root "./ellm_v1_data" --train \\
  --dataset "training_data/provided_sources/0000.parquet" \\
  --config "examples/training_config_lora_1epoch.json" \\
  --mapping "examples/mapping_question_answer.json"
```

Le chemin du checkpoint est un exemple à remplacer par le chemin local exact. Le profil d'une époque est un run initial configurable, pas un maximum architectural. Dans le conteneur de construction, le script a été vérifié par tests et son préflight a correctement déclaré `MISSING_DEPENDENCIES` (`transformers`/`safetensors` absents) sans charger de fixture ni prétendre à une inférence réelle. Le validateur sépare désormais `model_checkpoint_loaded` (un checkpoint local effectivement chargé) de `official_checkpoint_loaded` (poids dont le SHA-256 correspond à la révision officielle épinglée) : un modèle local d'identité personnalisée ne peut pas être étiqueté « officiel ». Le fine-tuning réel et l'évaluation réservée restent à exécuter sur le PC contenant les poids.

### 5.7 Expérience discriminante de résolution (P06)

Une expérience synthétique contrôlée, conservée dans `Amodel_eLLM_v1/experiments/P06_discriminating_ob` utilise cinq objets et cinq fonctions-réponses quantifiées. Les nombres de classes aux niveaux 0 à 5 sont $1, 2, 3, 3, 5, 5$, ce qui conserve des fibres non triviales aux niveaux 1–3. La capacité effective est 4 pour 5 sondes brutes et la dimension affine empirique vaut 4. Une opération ne factorise pas au niveau 1, mais factorise après ajout de la seconde sonde au niveau 2 ; une autre reste non factorisante au niveau 3. Les 125 triplets contrôlés respectent la loi du minimum double. Ces résultats sont exacts pour la matrice synthétique et sa quantification, pas une preuve de sémantique linguistique ni une mesure du SmolLM2.

# 6 Interprétation des résultats

La régression complète actuelle a passé 103/103 tests ; compilation Python, aides CLI runtime/installeur/validateur local, syntaxe JavaScript et smoke Chromium ont passé. Le smoke Chromium exécute 13/13 contrôles avec API simulée, et les routes HTTP sont testées séparément. Le test de dépendances du validateur côté PC renvoie le statut attendu `MISSING_DEPENDENCIES` dans ce conteneur, sans charger de poids fictifs. Les tests de checkpoint de cette suite utilisent des fixtures pour tester l'intégrité et le contrôle de hash, pas les poids officiels. Les tests unitaires montrent que certains contrats mathématiques et interfaces logicielles sont cohérents pour les domaines construits : signatures, préfixes, quantification, partition, fermeture et factorisation finies, seuils corrigés avec diagonale nulle, capacités/dimensions mesurées sur données fournies. L'adaptateur injecte effectivement une correction differentiable dans les logits d'un modèle causal de fixture et reçoit des gradients ; la fixture n'est utilisée que pour tester les composants et ne remplace jamais un checkpoint déployé.

Les tests d'import établissent la prise en charge des formats déclarés, y compris le Parquet réel de 7 473 lignes via le lecteur portable intégré. Les codecs et encodages Parquet hors du sous-ensemble documenté peuvent nécessiter PyArrow. Le code de fine-tuning est implémenté, y compris une perte causale pondérée par le nombre exact de tokens cibles dans chaque groupe d'accumulation (avec réduction FP32), les sauvegardes de reprise contenant paramètres entraînables/optimiseur/RNG et hash du checkpoint de base, le rechargement du meilleur epoch de validation et les tests d'ablations. Faute de poids réellement chargés, l'entraînement réel, la sélection réelle du checkpoint et les métriques linguistiques sur ce checkpoint ne sont pas mesurés ici. Le statut de l'ensemble est donc un logiciel v1 construit et partiellement testé, avec gate empirique bloqué par l'accès au checkpoint, et non un LLM entraîné/validé dans ce processus.

# 7 Conséquences théoriques, conceptuelles et opérationnelles

## 7.1 Théoriques

Les définitions de $\Phi$, empreintes, fibres, quotients, fermeture et factorisation sont appliquées à des états cachés et des sondes apprises. Les partitions et seuils quantifiés représentent le protocole déclaré et pas toute l'information continue. Le code trace explicitement le bord, sépare métrique sur identifiants et pseudoultramétrique sur quotient, et évite d'interpréter un succès empirique fini comme un théorème universel.

## 7.2 Conceptuelles

La fonction de l'empreinte diffère de v0 : les sondes apprises influencent le calcul de la prochaine distribution via un résidu de logits et sont supervisées à plusieurs échelles. Il faudra néanmoins évaluer si ces distinctions apprises correspondent à des invariants linguistiques transférables ; aucune sémantique n'est assignée aux sondes par construction.

## 7.3 Opérationnelles

L'utilisateur peut choisir des datasets, mapper les champs, régler le fine-tuning, suivre les jobs et interroger le modèle par une interface locale. Les données brutes et les erreurs sont conservées. L'historique SQLite permet recherche, export, renommage et suppression. Le checkpoint et les dépendances doivent être installés avant le chat génératif et l'entraînement effectifs.

# 8 Conclusions

Le principal correctif v1 par rapport à v0 est architectural : le modèle de langue de base doit être un vrai checkpoint causal préentraîné ; les sondes apprises participent aux logits et à la perte ;

la résolution structure l'empreinte ; l'import multi-format et le mapping de données sont intégrés à l'écran d'entraînement ; l'historique des conversations est persistant. La théorie n'est pas réduite au seul suffixe.

Les tests de composants ne valent pas une expérience d'entraînement avec le checkpoint réel. L'accès au dépôt officiel est resté bloqué lors de la construction des livrables ; la récupération et le fine-tuning réel restent une tâche objective à exécuter sur une machine connectée. Ce rapport ne prédit ni réussite ni échec futur : il sépare les mesures obtenues, les résultats non mesurés et les écarts précis.

# 9 Statut du modèle

— **Sources DDD_Base et datasets :** inventoriés, hashes/CRC vérifiés, originaux préservés.

— **Code du backend causal, sondes apprises et perte multi-échelle :** présent ; les contrats de composants sont testables sans poids externes.

— **Import, mapping, rapports et historique :** code présent et tests locaux en place ; le Parquet fourni fonctionne sans PyArrow, et d'autres variantes peuvent nécessiter ce paquet.

— **Interface Web/API :** présente ; tests HTTP exécutables sans checkpoint et refus explicite lorsque les poids manquent.

— **Téléchargement, chargement, fine-tuning et test sur checkpoint réel :** non vérifiés dans l'environnement de construction ; ne pas présenter comme exécutés.

— **Statut global :** livraison technique de eLLM_v1 avec un blocage empirique externe clairement tracé ; gate du modèle réel ouvert.